week - 1. Programming foundations

algorithm: a step by step procedure to solve a particular problem.

DS & A → Data Structures and Algorithms:

↳ How we process and solve problems using that data.

↳ How we process and solve problems using the data

Time complexity: describes how the no. of operations an algorithm performs grows as the input size increases.

e.g., to find ~~whether~~ whether an number exist in an array,

elements (n)	comparision
5	5
10	10
500	500
1000	1000

For this type of solution the time complexity is O(n).

↙

Because in the worst case, we might need to examine every elemen

Big O notation: a mathematical way of describing how an algorithm's performance grows when the input size increases.

~~t~~ complexity:

1) O(1) → ⊙ constant

⊙ Input size doesn't affect the no. of operations significantly.

e.g., arr = [10, 20, 30, 40, 50]
print (arr[2])

2. O(log n) → Logarithmic
   - The search space gets divided repeatedly by.
   - e.g. ~~merge sort~~ binary search
     - ~~it divides an array into smaller parts, sorts them, and merges the results.~~

3. O(n) → Linear
   - Operations grow proportionally to input ~~size~~
   - e.g., arr = [10, 20, 30, 40, 50]
     for i in arr:
         print (i)

4) O(n log n) → Linearithmic
   - A combinations of dividing and processing
   - Merge sort
     - it divides an array into smaller parts, sorts them and merges the results.

5) O(n^2) → Quadratic
   - Every elements may be compared with every other element.
   - arr = [10, 20, 30, 40, 50]
     for i in arr:
         for j in arr:
             print (i, j)

6) O(2^n) → Exponential
   - The no. of possibilities doubles with each additional input element.
   - Generating all subsets of an array.

Best case, average case and worst case.

consider searching for a no. in an array.

e.g. arr = [10, 20, 30, 40, 50]

here,

best case is finding 10
average case is finding 30
worst case is finding 50

note: In interviews, when someone asks for time complexity, you should generally analyse the ~~worst~~ worst case unless they ~~ask~~ specify otherwise.

space complexity: measures how the memory requirements grow.

e.g. 1

```
arr = [10, 20, 30, 40]
total = 0
for num in arr:
    total += num
print(total)
```

we only use one additional variable, total, regardless of array size.

e.g. 2

```
~~total = 0~~
arr = [10, 20, 30, 40]
~~total = 0~~
new_arr = []
for num in arr:
    new_arr.append(num*2)
print(new_arr)
```

we create another array whose size growth with the input.

if we need	we need ___ auxiliary space
one extra variable	$O(1)$
New array of n elements	$O(n)$
2D matrix of $n \times n$	$O(n^2)$

Big O rules for leetcode :

1) Ignore constants.

e.g. for i in range(n):
print(i)

for i in range(n):
print(i)

$\Rightarrow O(n) + O(n) = O(n)$

2) Nested loops multiply:

e.g. for i in range(n):
for j in range(n):
print(i, j)

$\Rightarrow O(n^2)$

3) Different input sizes:

e.g. for i in range(n):
print(i)

for j in range(m):
print(j)

4) Drop lower order terms.

$\Rightarrow O(n+m)$, not $O(n)$.

if $O(n^2 + n + 5) \Rightarrow O(n^2)$

Day 1 Basics of array and its traversal.

1480 = Running sum of 1d array.

⇒ normally we will use the naive approach, also called Brute Force method.

⇒ Brute Force method.

arr = [1, 2, 3, 4, 5].

iteration 1

arr[0] = [1]

arr[1] = [1+2]

arr[2] = [1+2+3]

arr[3] = [1+2+3+4]

goes on.

we will be using a nested loop to find the sum of previous elements each time.

Time complexity: $O(n^2)$

Space complexity: $O(1)$ → ~~extra space if you modify the input array in place.~~

$O(n)$ → ~~if a new array is created to store the values.~~

⇒ The Optimized Pattern: Prefix Sum.

~~if we add the previous element~~

if we want to find the previous elements sum of arr[i], it will be very easy if the sum is at arr[i-1].

so that, arr[i] will get substituted by,

$$arr[i] = arr[i-1] + arr[i].$$

code:

```
class Solution(object):
    def runningSum(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """
        for i in range(1, len(nums)):
            nums[i] = nums[i-1] + nums[i]
        return nums.
```

complexities:

Time: O(n) - Single pass through the list

space: O(1) - Modified in-place without extra space.

1672 = Richest Customer wealth:

a matrix of list of list is given. where ~~the no of~~ each column represents each customer, and each row represents ~~the amount of customer has in each bank~~ the amount of money the customer has in each bank.

solution: very normal.

add the row values → compare with previous ones → move ahead.

code.

```
class Solution(object):
    def maximumWealth(self, accounts):
        """
        :type accounts: List[List[int]]
        :rtype: int
        """
        max_wealth = 0
        for i in range(0, len(accounts)):
            sum = 0
            for j in range(0, len(accounts[0])
                sum = 0
                for j in range(0, len(accounts[0]
                    sum = sum + accounts[i][j]
                if max_wealth < sum:
                    max_wealth = sum
        return max_wealth.
```

Time complexity : $O(m \times n)$

space " : $O(1)$.